

[↶ ENTERPRISE 3-TIER ARCHITECTURE](#)

Shop Management System

Complete Architecture, Active Source Code, 100% Stored Procedure Database Setup, and Deep Interactive Data Flow Guide.

 ASP.NET Core MVC (.NET 10) MySQL 8.0 (19 Stored Procedures) 3-Tier (DAL + BAL + Web) ClosedXML Bulk Stream Pessimistic Locking (FOR UPDATE) Clean wwwroot/js/ Separation

 Search methods, Stored Procedures (e.g. sp_CheckProductStock), classes, or files...

[⚡ INTERACTIVE FLOW INSPECTOR](#)

Project Workflow & Deep Data Lifecycle Explorer

Select any module below to inspect the step-by-step pipeline from Client UI to MySQL Stored Procedure and Table Commit.

CHOOSE FEATURE PIPELINE:

 **1. Invoice Checkout & Atomic Stock Deduction (POS)** ▾

1

Presentation

2

Network & HTTP Route

3

Presentation Controller

4

Business Access Layer (BAL)

5

Data Access Layer (DAL)

6

Database Layer (MySQL)

STEP 1 OF 6

PRESENTATION / CLIENT UI

 ShopManagement.Web/wwwroot/js/invoice-create.js

Cashier adds products & clicks 'Complete Sale'

English: The user builds the cart in the browser. When clicking 'Complete Sale', JavaScript validates customer name and non-empty cart, serializes items into a JSON string, and sends an AJAX POST request.

Hinglish: Cashier products add karta hai cart mein. Jab 'Complete Sale' dabata hai, JavaScript validation karta hai (Customer Name required, cart empty nahi honi chahiye) aur pura cart JSON string mein pack karke AJAX POST request bhejta hai.

↶ DATA PAYLOAD / SIGNATURE:

```
customerName=Rajesh Kumar&customerPhone=9876543210&itemsJson=[{"productId":1,"productName":"Parker Pen","quantity":2,"unitPrice":150.00}]
```

</> ACTIVE CODE SECTION:

Next →

← Prev

```
$("#saveInvoiceBtn").on("click", function () {
    var custName = $("#customerName").val().trim();
    var custPhone = $("#customerPhone").val().trim();

    if (!custName) {
        Swal.fire({ icon: 'warning', title: 'Customer Name Required',
        return;
    }
    if (cart.length === 0) {
        Swal.fire({ icon: 'warning', title: 'Cart Empty', text: 'Pleas
        return;
    }

    var postData = {
        customerName: custName,
        customerPhone: custPhone,
        itemsJson:     JSON.stringify(cart)
    };
});
```



2. Master Database Setup

(`ShopManagement\_Full\_Setup.sql`)

The entire database schema, relationship constraints, default administrator / branch seed records, and all **19 production Stored Procedures** are defined centrally in a single master script. Execute this script once in MySQL Workbench, CLI, or phpMyAdmin to initialize everything cleanly.

```
-- =====
-- SHOP MANAGEMENT SYSTEM - COMPLETE MASTER DATABASE & STORED PROC
-- =====
-- Database Engine : MySQL 8.0+ / MariaDB 10.4+
-- Character Set   : utf8mb4 (Full Unicode / Emoji / Multilingual
-- Collation       : utf8mb4_unicode_ci
-- Execution Order : Run this script once on a fresh or existing I
--                  It creates the database, all tables, indexes,
--                  and all 19 Stored Procedures required by the
-- =====

-- -----
-- STEP 1: CREATE & SELECT DATABASE
-- -----

CREATE DATABASE IF NOT EXISTS ShopManagementDB
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE ShopManagementDB;
```

Copy SQL



3. Data Access Layer

(ShopManagement.DataAccess)

Contains lightweight ADO.NET abstractions using `MySQLConnector`. All database interactions call compiled Stored Procedures with strongly typed parameters.

3.1 DbHelper.cs (Clean, High-Performance ADO.NET Helper)

Zero embedded Stored Procedure SQL strings. Clean execution overloads supporting transactions and Stored Procedures.

```
using System;
using System.Data;
using MySqlConnector;

namespace ShopManagement.DataAccess
{
    /// <summary>
    /// Lightweight, high-performance database helper for MySQL operations.
    /// Executes plain SQL text and Stored Procedures via ADO.NET and MySQLCon
    /// Stored Procedure definitions are maintained centrally in the master SQ
    /// </summary>
    public static class DbHelper
    {
        public static string ConnectionString { get; set; } = "Server=localhost

        public static MySqlConnection GetConnection()
        {
            var conn = new MySqlConnection(ConnectionString);
            if (conn.State != ConnectionState.Open)
            {
                conn.Open();
            }
            return conn;
        }

        public static DataTable ExecuteDataTable(string query, MySqlParameter[]
        {
            var dt = new DataTable();
            using (var conn = GetConnection())
            using (var cmd = new MySqlCommand(query, conn))
            {
                cmd.CommandType = CommandType;
                if (parameters != null && parameters.Length > 0)
                {
                    cmd.Parameters.AddRange(parameters);
                }

                using (var adapter = new MySqlDataAdapter(cmd))
                {
                    adapter.Fill(dt);
                }
            }
            return dt;
        }
    }
}
```

```
}

public static int ExecuteNonQuery(string query, MySqlParameter[] param
{
    using (var conn = GetConnection())
    using (var cmd = new MySqlCommand(query, conn))
    {
        cmd.CommandType = commandType;
        if (parameters != null && parameters.Length > 0)
        {
            cmd.Parameters.AddRange(parameters);
        }
        return cmd.ExecuteNonQuery();
    }
}

public static object ExecuteScalar(string query, MySqlParameter[] para
{
    using (var conn = GetConnection())
    using (var cmd = new MySqlCommand(query, conn))
    {
        cmd.CommandType = commandType;
        if (parameters != null && parameters.Length > 0)
        {
            cmd.Parameters.AddRange(parameters);
        }
        return cmd.ExecuteScalar();
    }
}

#region Stored Procedure Convenience Helpers

public static DataTable ExecuteStoredProcedureDataTable(string spName,
{
    return ExecuteDataTable(spName, parameters, CommandType.StoredProc
}

public static int ExecuteStoredProcedureNonQuery(string spName, MySQLP
{
    return ExecuteNonQuery(spName, parameters, CommandType.StoredProce
}

public static object ExecuteStoredProcedureScalar(string spName, MySQL
{
    return ExecuteScalar(spName, parameters, CommandType.StoredProcedu
}
```

```

#endregion

/// <summary>
/// Verifies database connectivity on application startup.
/// Table schema, stored procedures, and triggers are deployed via the
/// </summary>
public static void InitializeDatabase()
{
    try
    {
        using (var conn = GetConnection())
        {
            // Simple connectivity test
            using (var cmd = new MySqlCommand("SELECT 1;", conn))
            {
                cmd.ExecuteScalar();
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[DbHelper.InitializeDatabase] Warning: {ex}");
    }
}
}

```

3.2 InvoiceDAL.cs (ACID Billing Transaction & Pessimistic Lock)

Executes `sp_CheckProductStock` with `FOR UPDATE` lock, `sp_InsertInvoiceHeader`, `sp_InsertInvoiceDetail`, and `sp_DeductProductStock` within a shared C# database transaction.

```

using System;
using System.Data;
using MySqlConnector;

namespace ShopManagement.DataAccess
{
    public class InvoiceDAL
    {
        public bool CreateInvoice(string customerName, string customerPhone, i
        {
            invoiceNo    = "INV-" + DateTime.Now.ToString("yyyyMMddHHmmss");

```

```
errorMessage = string.Empty;

using (var conn = DbHelper.GetConnection())
using (var trans = conn.BeginTransaction())
{
    try
    {
        decimal grandTotal = 0;

        // — Step 1: Validate stock availability for every item (
        foreach (DataRow row in items.Rows)
        {
            int    productId = Convert.ToInt32(row["ProductId"]);
            int     qty       = Convert.ToInt32(row["Quantity"]);
            decimal unitPrice = Convert.ToDecimal(row["UnitPrice"]);
            grandTotal += qty * unitPrice;

            using (var cmd = new MySqlCommand("sp_CheckProductStoc
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.AddWithValue("p_ProductId", product

            using (var reader = cmd.ExecuteReader())
            {
                if (!reader.Read())
                {
                    reader.Close();
                    errorMessage = $"Product ID {productId} do
                    trans.Rollback();
                    return false;
                }

                string prodName    = reader["ProductName"].ToS
                int    stockOnHand = Convert.ToInt32(reader["Q
                reader.Close();

                if (stockOnHand < qty)
                {
                    errorMessage = $"Insufficient stock for '{
                    trans.Rollback();
                    return false;
                }
            }
        }
    }
}
```

// — Step 2: Insert Invoice Header → get new InvoiceId —

```
int invoiceId;
using (var cmd = new MySqlCommand("sp_InsertInvoiceHeader"
{
    cmd.CommandType = CommandType.StoredProcedure;
    cmd.Parameters.AddWithValue("p_InvoiceNo",    invoice
    cmd.Parameters.AddWithValue("p_CustomerName", custome
    cmd.Parameters.AddWithValue("p_CustomerPhone", string.
    cmd.Parameters.AddWithValue("p_CreatedBy",    created
    cmd.Parameters.AddWithValue("p_TotalAmount",  grandTo

    using (var reader = cmd.ExecuteReader())
    {
        reader.Read();
        invoiceId = Convert.ToInt32(reader["NewInvoiceId"])
    }
}

// — Step 3: Insert each detail row and deduct stock —
foreach (DataRow row in items.Rows)
{
    int    productId = Convert.ToInt32(row["ProductId"]);
    string prodName  = Convert.ToString(row["ProductName"]);
    int    qty       = Convert.ToInt32(row["Quantity"]);
    decimal unitPrice = Convert.ToDecimal(row["UnitPrice"]);
    decimal lineTotal = qty * unitPrice;

    // 3a. Insert detail line
    using (var cmd = new MySqlCommand("sp_InsertInvoiceDet
    {
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("p_InvoiceId",    invoi
        cmd.Parameters.AddWithValue("p_ProductId",    produ
        cmd.Parameters.AddWithValue("p_ProductName",  prodN
        cmd.Parameters.AddWithValue("p_Quantity",     qty);
        cmd.Parameters.AddWithValue("p_UnitPrice",    unitP
        cmd.Parameters.AddWithValue("p_TotalAmount",  lineT
        cmd.ExecuteNonQuery();
    }

    // 3b. Deduct stock
    using (var cmd = new MySqlCommand("sp_DeductProductSto
    {
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("p_ProductId", product
        cmd.Parameters.AddWithValue("p_Quantity",  qty);
        cmd.ExecuteNonQuery();
    }
```

```
        }

        trans.Commit();
        return true;
    }
    catch (Exception ex)
    {
        trans.Rollback();
        errorMessage = ex.Message;
        return false;
    }
}

}

public DataTable GetInvoiceHistory(int? createdBy = null)
{
    var parameters = new[]
    {
        new MySQLParameter("@p_CreatedBy", createdBy.HasValue ? (object)
    };
    return DbHelper.ExecuteDataTable("sp_GetInvoiceHistory", parameter

}

public DataTable GetInvoiceHeader(int invoiceId)
{
    var parameters = new[] { new MySQLParameter("@p_InvoiceId", invoic
    return DbHelper.ExecuteDataTable("sp_GetInvoiceHeader", parameters
}

public DataTable GetInvoiceDetails(int invoiceId)
{
    var parameters = new[] { new MySQLParameter("@p_InvoiceId", invoic
    return DbHelper.ExecuteDataTable("sp_GetInvoiceDetails", parameter
}

public DataTable GetDashboardMetrics()
{
    return DbHelper.ExecuteDataTable("sp_GetDashboardMetrics", null, C
}

}
```

3.3 ProductDAL.cs

```
using System;
using System.Data;
using MySqlConnector;

namespace ShopManagement.DataAccess
{
    public class ProductDAL
    {
        public DataTable GetAllProducts(bool activeOnly = true)
        {
            var parameters = new[] { new MySqlParameter("@p_ActiveOnly", activeOnly) };
            return DbHelper.ExecuteDataTable("sp_GetAllProducts", parameters,
            }

        public DataTable GetProductById(int productId)
        {
            var parameters = new[] { new MySqlParameter("@p_ProductId", productId) };
            return DbHelper.ExecuteDataTable("sp_GetProductById", parameters,
            }

        public DataTable CheckProductExists(string productCode, string productName)
        {
            var parameters = new[]
            {
                new MySqlParameter("@p_ProductCode", productCode ?? string.Empty),
                new MySqlParameter("@p_ProductName", productName ?? string.Empty)
            };
            return DbHelper.ExecuteDataTable("sp_CheckProductExists", parameters,
            }

        public bool UpsertProduct(string productCode, string productName, int quantity)
        {
            var dtExisting = CheckProductExists(productCode, productName);

            if (dtExisting != null && dtExisting.Rows.Count > 0)
            {
                int existingId = Convert.ToInt32(dtExisting.Rows[0]["ProductId"]);
                string existingImage = Convert.ToString(dtExisting.Rows[0]["ImagePath"]);
                int currentQty = Convert.ToInt32(dtExisting.Rows[0]["Quantity"]);

                string resolvedImage = string.IsNullOrEmpty(imagePath) ? existingImage : imagePath;

                var updateParams = new[]
                {
                    new MySqlParameter("@p_ProductId", existingId),
                    new MySqlParameter("@p_ProductName", productName),
                    new MySqlParameter("@p_Quantity", quantity),
                    new MySqlParameter("@p_ResolvedImagePath", resolvedImage)
                };
                return DbHelper.ExecuteNonQuery("sp_UpsertProduct", updateParams,
            }
        }
    }
}
```

```
        new MySqlParameter("@p_Quantity",    currentQty + quantity),
        new MySqlParameter("@p_Price",      price),
        new MySqlParameter("@p_ImagePath",   resolvedImage ?? "/images/products/default-product.svg");

    };

    int rows = DbHelper.ExecuteNonQuery("sp_UpdateProduct", update
    operation = "Updated (Stock Incremented)");
    return rows > 0;
}
else
{
    string defaultImage = string.IsNullOrEmpty(imagePath)
        ? "/images/products/default-product.svg"
        : imagePath;

    var insertParams = new[]
    {
        new MySqlParameter("@p_ProductCode", productCode),
        new MySqlParameter("@p_ProductName", productName),
        new MySqlParameter("@p_Quantity",    quantity),
        new MySqlParameter("@p_Price",      price),
        new MySqlParameter("@p_ImagePath",   defaultImage)
    };

    int rows = DbHelper.ExecuteNonQuery("sp_CreateProduct", insert
    operation = "Created New Product");
    return rows > 0;
}

}

public bool UpdateProduct(int productId, string productName, int quant
{
    errorMessage = string.Empty;
    try
    {
        var parameters = new[]
        {
            new MySqlParameter("@p_ProductId",    productId),
            new MySqlParameter("@p_ProductName",   productName),
            new MySqlParameter("@p_Quantity",      quantity),
            new MySqlParameter("@p_Price",         price),
            new MySqlParameter("@p_ImagePath",     string.IsNullOrEmpty
        );

        return DbHelper.ExecuteNonQuery("sp_UpdateProduct", parameters
    }
    catch (Exception ex)
```

```
        {
            errorMessage = ex.Message;
            return false;
        }
    }

    public bool DeleteProduct(int productId)
    {
        var parameters = new[] { new MySqlParameter("@p_ProductId", productId) };
        return DbHelper.ExecuteNonQuery("sp_DeleteProduct", parameters, CommandType.StoredProcedure);
    }

    public DataTable SearchAvailableProducts(string term)
    {
        var parameters = new[] { new MySqlParameter("@p_Term", term ?? string.Empty) };
        return DbHelper.ExecuteDataTable("sp_SearchAvailableProducts", parameters, CommandType.StoredProcedure);
    }
}
```

3.4 EmployeeDAL.cs

```
using System;
using System.Data;
using MySqlConnector;

namespace ShopManagement.DataAccess
{
    public class EmployeeDAL
    {
        public DataTable GetAllEmployees()
        {
            return DbHelper.ExecuteDataTable("sp_GetAllEmployees", null, CommandType.StoredProcedure);
        }

        public DataTable GetEmployeeById(int userId)
        {
            var parameters = new[] { new MySqlParameter("@p_UserId", userId) };
            return DbHelper.ExecuteDataTable("sp_GetEmployeeById", parameters, CommandType.StoredProcedure);
        }

        public bool CreateEmployee(string name, string username, string password)
        {
            var parameters = new[] { new MySqlParameter("@p_Name", name), new MySqlParameter("@p_Username", username), new MySqlParameter("@p_Password", password) };
            return DbHelper.ExecuteNonQuery("sp_CreateEmployee", parameters, CommandType.StoredProcedure) > 0;
        }
    }
}
```

```
{
    new SqlParameter("@p_Name", name),
    new SqlParameter("@p_Username", username),
    new SqlParameter("@p_Password", password),
    new SqlParameter("@p_StoreId", storeId.HasValue ? (object)s
};
return DbHelper.ExecuteNonQuery("sp_CreateEmployee", parameters, C
}

public bool UpdateEmployee(int userId, string name, string password, b
{
    var parameters = new[]
    {
        new SqlParameter("@p_UserId", userId),
        new SqlParameter("@p_Name", name),
        new SqlParameter("@p_Password", string.IsNullOrEmpty(pa
        new SqlParameter("@p_IsActive", isActive ? 1 : 0),
        new SqlParameter("@p_StoreId", storeId.HasValue ? (object)s
    };
    return DbHelper.ExecuteNonQuery("sp_UpdateEmployee", parameters, C
}

public bool ResetPassword(int userId, string newPassword)
{
    var parameters = new[]
    {
        new SqlParameter("@p_UserId", userId),
        new SqlParameter("@p_Password", newPassword)
    };
    return DbHelper.ExecuteNonQuery("sp_ResetEmployeePassword", paramet
}

public bool ToggleStatus(int userId)
{
    var parameters = new[] { new SqlParameter("@p_UserId", userId) }
    return DbHelper.ExecuteNonQuery("sp_ToggleEmployeeStatus", paramet
}

public bool DeleteEmployee(int userId)
{
    var parameters = new[] { new SqlParameter("@p_UserId", userId) }
    return DbHelper.ExecuteNonQuery("sp_DeleteEmployee", parameters, C
}
}
```

3.5 StoreDAL.cs

```
using System;
using System.Data;
using MySqlConnector;

namespace ShopManagement.DataAccess
{
    public class StoreDAL
    {
        public DataTable GetAllStores(bool activeOnly = false)
        {
            var parameters = new[] { new MySqlParameter("@p_ActiveOnly", activeOnly) };
            return DbHelper.ExecuteDataTable("sp_GetAllStores", parameters, Co

        }

        public DataTable GetStoreById(int storeId)
        {
            var parameters = new[] { new MySqlParameter("@p_StoreId", storeId) };
            return DbHelper.ExecuteDataTable("sp_GetStoreById", parameters, Co

        }

        public bool CreateStore(string storeCode, string storeName, string address,
        {
            errorMessage = string.Empty;
            try
            {
                var parameters = new[]
                {
                    new MySqlParameter("@p_StoreCode", storeCode),
                    new MySqlParameter("@p_StoreName", storeName),
                    new MySqlParameter("@p_Address", address, string.IsNullOrEmpty(address) ? string.Empty : address),
                    new MySqlParameter("@p_Phone", phone, string.IsNullOrEmpty(phone) ? string.Empty : phone),
                    new MySqlParameter("@p_Email", email, string.IsNullOrEmpty(email) ? string.Empty : email)
                };
                return DbHelper.ExecuteNonQuery("sp_CreateStore", parameters,

            }
            catch (Exception ex)
            {
                errorMessage = ex.Message.Contains("Duplicate") ? "Store code already exists" : ex.Message;
                return false;
            }
        }

        public bool UpdateStore(int storeId, string storeName, string address,
        {
```

```
var parameters = new[]
{
    new MySqlParameter("@p_StoreId", storeId),
    new MySqlParameter("@p_StoreName", storeName),
    new MySqlParameter("@p_Address", string.IsNullOrEmpty(a
    new MySqlParameter("@p_Phone", string.IsNullOrEmpty(p
    new MySqlParameter("@p_Email", string.IsNullOrEmpty(e
    new MySqlParameter("@p_IsActive", isActive ? 1 : 0)
};
return DbHelper.ExecuteNonQuery("sp_UpdateStore", parameters, Comm
}

public bool DeleteStore(int storeId)
{
    var parameters = new[] { new MySqlParameter("@p_StoreId", storeId)
    return DbHelper.ExecuteNonQuery("sp_DeleteStore", parameters, Comm
}

public DataTable GetStoreStats()
{
    return DbHelper.ExecuteDataTable("sp_GetStoreStats", null, Command
}
}
```

3.6 AccountDAL.cs

```
using System.Data;
using MySqlConnector;

namespace ShopManagement.DataAccess
{
    public class AccountDAL
    {
        public DataTable ValidateUser(string username, string password)
        {
            var parameters = new[]
            {
                new MySqlParameter("@p_Username", username),
                new MySqlParameter("@p_Password", password)
            };

            return DbHelper.ExecuteDataTable("sp_ValidateUser", parameters, Co
        }
    }
}
```

```
}  
}
```

4. Business Access Layer (ShopManagement.Business)

Validates data integrity, enforces security rules, handles upsert calculation logic, and streams ClosedXML Excel data.

4.1 InvoiceBAL.cs

```
using System;  
using System.Data;  
using ShopManagement.DataAccess;  
  
namespace ShopManagement.Business  
{  
    public class InvoiceBAL  
    {  
        private readonly InvoiceDAL _dal = new InvoiceDAL();  
  
        public bool CreateInvoice(string customerName, string customerPhone, i  
        {  
            invoiceNo = string.Empty;  
            errorMessage = string.Empty;  
  
            if (string.IsNullOrEmpty(customerName))  
            {  
                errorMessage = "Customer Name is required.";   
                return false;  
            }  
  
            if (items == null || items.Rows.Count == 0)  
            {  
                errorMessage = "Invoice must contain at least one product item  
                return false;  
            }  
        }  
    }  
}
```

```
        return _dal.CreateInvoice(customerName.Trim(), customerPhone?.Trim()
    }

    public DataTable GetInvoiceHistory(int? createdBy = null)
    {
        return _dal.GetInvoiceHistory(createdBy);
    }

    public DataTable GetInvoiceHeader(int invoiceId)
    {
        return _dal.GetInvoiceHeader(invoiceId);
    }

    public DataTable GetInvoiceDetails(int invoiceId)
    {
        return _dal.GetInvoiceDetails(invoiceId);
    }

    public DataTable GetDashboardMetrics()
    {
        return _dal.GetDashboardMetrics();
    }
}
}
```

4.2 ProductBAL.cs

```
using System;
using System.Data;
using ShopManagement.DataAccess;

namespace ShopManagement.Business
{
    public class ProductBAL
    {
        private readonly ProductDAL _dal = new ProductDAL();

        public DataTable GetAllProducts(bool activeOnly = true)
        {
            return _dal.GetAllProducts(activeOnly);
        }

        public DataTable GetProductById(int productId)
        {
            return _dal.GetProductById(productId);
        }
    }
}
```

```
{
    return _dal.GetProductById(productId);
}

/// <summary>
/// Upsert: If product already exists (by code OR name), increment its
/// If not found, create a new product. Missing image defaults to defa
/// </summary>
public bool UpsertProduct(string productCode, string productName, int
{
    if (string.IsNullOrEmpty(productCode) || string.IsNullOrEmpty
    {
        operation = "Product code and name are required.";
        return false;
    }

    if (quantity < 0) quantity = 0;
    if (price < 0) price = 0;

    return _dal.UpsertProduct(productCode.Trim(), productName.Trim(),
}

/// <summary>
/// Edit existing product: sets exact quantity, price, name, and optio
/// </summary>
public bool UpdateProduct(int productId, string productName, int quant
{
    errorMessage = string.Empty;

    if (string.IsNullOrEmpty(productName))
    {
        errorMessage = "Product Name is required.";
        return false;
    }

    if (quantity < 0) quantity = 0;
    if (price < 0) price = 0;

    return _dal.UpdateProduct(productId, productName.Trim(), quantity,
}

public bool DeleteProduct(int productId)
{
    return _dal.DeleteProduct(productId);
}

public DataTable SearchAvailableProducts(string term)
```

```
        {  
            return _dal.SearchAvailableProducts(term);  
        }  
    }  
}
```

4.3 EmployeeBAL.cs

```
using System;  
using System.Data;  
using ShopManagement.DataAccess;  
  
namespace ShopManagement.Business  
{  
    public class EmployeeBAL  
    {  
        private readonly EmployeeDAL _dal = new EmployeeDAL();  
  
        public DataTable GetAllEmployees()  
        {  
            return _dal.GetAllEmployees();  
        }  
  
        public DataTable GetEmployeeById(int userId)  
        {  
            return _dal.GetEmployeeById(userId);  
        }  
  
        public bool CreateEmployee(string name, string username, string password)  
        {  
            errorMessage = string.Empty;  
            if (string.IsNullOrEmpty(name) || string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))  
            {  
                errorMessage = "All fields (Name, Username, Password) are required";  
                return false;  
            }  
  
            try  
            {  
                return _dal.CreateEmployee(name.Trim(), username.Trim(), password.Trim());  
            }  
            catch (Exception ex)  
            {  
                errorMessage = ex.Message.Contains("Duplicate") ? ex.Message : "An error occurred while creating the employee.";  
                return false;  
            }  
        }  
    }  
}
```

```
        ? "Username already exists. Please choose a different user  
        : ex.Message;  
        return false;  
    }  
}  
  
public bool UpdateEmployee(int userId, string name, string password, b  
{  
    errorMessage = string.Empty;  
    if (string.IsNullOrEmpty(name))  
    {  
        errorMessage = "Name is required.";   
        return false;  
    }  
  
    try  
    {  
        return _dal.UpdateEmployee(userId, name.Trim(), password?.Trim  
    }  
    catch (Exception ex)  
    {  
        errorMessage = ex.Message;  
        return false;  
    }  
}  
  
public bool ResetPassword(int userId, string newPassword, out string e  
{  
    errorMessage = string.Empty;  
    if (string.IsNullOrEmpty(newPassword))  
    {  
        errorMessage = "New password cannot be empty.";   
        return false;  
    }  
  
    try  
    {  
        return _dal.ResetPassword(userId, newPassword.Trim());  
    }  
    catch (Exception ex)  
    {  
        errorMessage = ex.Message;  
        return false;  
    }  
}  
  
public bool ToggleStatus(int userId)
```

```
{  
    return _dal.ToggleStatus(userId);  
}  
  
public bool DeleteEmployee(int userId)  
{  
    return _dal.DeleteEmployee(userId);  
}  
}  
}
```

4.4 StoreBAL.cs

```
using System;  
using System.Data;  
using ShopManagement.DataAccess;  
  
namespace ShopManagement.Business  
{  
    public class StoreBAL  
    {  
        private readonly StoreDAL _dal = new StoreDAL();  
  
        public DataTable GetAllStores(bool activeOnly = false)  
        {  
            return _dal.GetAllStores(activeOnly);  
        }  
  
        public DataTable GetStoreById(int storeId)  
        {  
            return _dal.GetStoreById(storeId);  
        }  
  
        public DataTable GetStoreStats()  
        {  
            return _dal.GetStoreStats();  
        }  
  
        public bool CreateStore(string storeCode, string storeName, string add  
        {  
            errorMessage = string.Empty;  
  
            if (string.IsNullOrEmpty(storeCode) || string.IsNullOrEmptySp  
            {
```

```
        errorMessage = "Store Code and Store Name are required.";
        return false;
    }

    return _dal.CreateStore(storeCode.Trim().ToUpper(), storeName.Trim()
}

public bool UpdateStore(int storeId, string storeName, string address,
{
    errorMessage = string.Empty;

    if (string.IsNullOrEmpty(storeName))
    {
        errorMessage = "Store Name is required.";
        return false;
    }

    return _dal.UpdateStore(storeId, storeName.Trim(), address, phone,
}

public bool DeleteStore(int storeId)
{
    return _dal.DeleteStore(storeId);
}
}
}
```

4.5 AccountBAL.cs

```
using System;
using System.Data;
using ShopManagement.DataAccess;

namespace ShopManagement.Business
{
    public class AccountBAL
    {
        private readonly AccountDAL _dal = new AccountDAL();

        public DataTable Login(string username, string password)
        {
            if (string.IsNullOrEmpty(username) || string.IsNullOrEmptyPa
            {
                return new DataTable();
            }
        }
    }
}
```

```
    }  
  
    return _dal.ValidateUser(username.Trim(), password.Trim());  
}  
}  
}
```

4.6 ExcelBAL.cs

```
using System;  
using System.Data;  
using System.IO;  
using ClosedXML.Excel;  
  
namespace ShopManagement.Business  
{  
    public class ExcelBAL  
    {  
        private readonly ProductBAL _productBAL = new ProductBAL();  
  
        public DataTable ProcessExcelProductImport(Stream fileStream, out int  
        {  
            totalCount = 0;  
            insertedCount = 0;  
            updatedCount = 0;  
            failedCount = 0;  
  
            var resultDt = new DataTable("ImportResults");  
            resultDt.Columns.Add("RowNumber", typeof(int));  
            resultDt.Columns.Add("ProductCode", typeof(string));  
            resultDt.Columns.Add("ProductName", typeof(string));  
            resultDt.Columns.Add("Quantity", typeof(int));  
            resultDt.Columns.Add("Price", typeof(decimal));  
            resultDt.Columns.Add("Status", typeof(string));  
            resultDt.Columns.Add("Message", typeof(string));  
  
            using (var workbook = new XLWorkbook(fileStream))  
            {  
                var worksheet = workbook.Worksheet(1);  
                var rows = worksheet.RangeUsed().RowsUsed();  
  
                bool isFirstRow = true;  
                int rowNum = 1;
```

```
foreach (var row in rows)
{
    if (isFirstRow)
    {
        // Header row: ProductCode | ProductName | Quantity |
        isFirstRow = false;
        rowNum++;
        continue;
    }

    totalCount++;
    string productCode = row.Cell(1).GetString()?.Trim();
    string productName = row.Cell(2).GetString()?.Trim();
    string qtyStr = row.Cell(3).GetString()?.Trim();
    string priceStr = row.Cell(4).GetString()?.Trim();
    string imagePath = row.Cell(5).GetString()?.Trim();

    if (string.IsNullOrEmpty(productCode) && string.IsNul
    {
        // empty row
        continue;
    }

    int qty = 0;
    decimal price = 0;
    bool isQtyValid = int.TryParse(qtyStr, out qty);
    bool isPriceValid = decimal.TryParse(priceStr, out price);

    if (string.IsNullOrEmpty(productCode) || string.IsNul
    {
        failedCount++;
        resultDt.Rows.Add(rowNum, productCode, productName, qt
        rowNum++;
        continue;
    }

    string operation;
    bool success = _productBAL.UpsertProduct(productCode, prod

    if (success)
    {
        if (operation.Contains("Updated"))
            updatedCount++;
        else
            insertedCount++;

        resultDt.Rows.Add(rowNum, productCode, productName, qt
```

```
        }
        else
        {
            failedCount++;
            resultDt.Rows.Add(rowNum, productCode, productName, qt
        }

        rowNum++;
    }
}

return resultDt;
}

public byte[] GenerateSampleExcelTemplate()
{
    using (var workbook = new XLWorkbook())
    {
        var ws = workbook.Worksheets.Add("Products");

        // Headers
        ws.Cell(1, 1).Value = "ProductCode";
        ws.Cell(1, 2).Value = "ProductName";
        ws.Cell(1, 3).Value = "Quantity";
        ws.Cell(1, 4).Value = "Price";
        ws.Cell(1, 5).Value = "ImagePath";

        // Header styling
        var headerRange = ws.Range("A1:E1");
        headerRange.Style.Font.Bold = true;
        headerRange.Style.Fill.BackgroundColor = XLColor.FromHtml("#25
        headerRange.Style.Font.FontColor = XLColor.White;

        // Sample data rows
        ws.Cell(2, 1).Value = "P101";
        ws.Cell(2, 2).Value = "Wireless Optical Mouse";
        ws.Cell(2, 3).Value = 25;
        ws.Cell(2, 4).Value = 499.00;
        ws.Cell(2, 5).Value = "/images/products/mouse.png";

        ws.Cell(3, 1).Value = "P102";
        ws.Cell(3, 2).Value = "Mechanical Gaming Keyboard";
        ws.Cell(3, 3).Value = 15;
        ws.Cell(3, 4).Value = 1899.00;
        ws.Cell(3, 5).Value = "/images/products/keyboard.png";

        ws.Cell(4, 1).Value = "P001"; // Test existing upsert
```

```

ws.Cell(4, 2).Value = "Parker Roller Ball Pen";
ws.Cell(4, 3).Value = 50; // Will be added to existing stock
ws.Cell(4, 4).Value = 150.00;
ws.Cell(4, 5).Value = "/images/products/pen.png";

ws.Columns().AdjustToContents();

using (var ms = new MemoryStream())
{
    workbook.SaveAs(ms);
    return ms.ToArray();
}
}
}
}
}

```



5. Presentation Layer Controllers (ShopManagement.Web)

5.1 InvoiceController.cs

```

using System;
using System.Data;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Newtonsoft.Json;
using Newtonsoft.Json.Linq;
using ShopManagement.Business;

namespace ShopManagement.Web.Controllers
{
    public class InvoiceController : Controller
    {
        private readonly InvoiceBAL _invoiceBAL = new InvoiceBAL();
        private readonly ProductBAL _productBAL = new ProductBAL();
    }
}

```

```
private bool IsLoggedIn()
{
    return !string.IsNullOrEmpty(HttpContext.Session.GetString("Role"))
}

[HttpGet]
public IActionResult Create()
{
    if (!IsLoggedIn()) return RedirectToAction("Login", "Account");
    return View();
}

[HttpPost]
public IActionResult Create(string customerName, string customerPhone,
{
    if (!IsLoggedIn()) return Json(new { success = false, message = "S

    if (string.IsNullOrEmpty(customerName))
    {
        return Json(new { success = false, message = "Customer name is
    }

    if (string.IsNullOrEmpty(itemsJson))
    {
        return Json(new { success = false, message = "Please select at
    }

    try
    {
        var jArray = JArray.Parse(itemsJson);
        if (jArray.Count == 0)
        {
            return Json(new { success = false, message = "Invoice must
        }

        var itemsDt = new DataTable("InvoiceItems");
        itemsDt.Columns.Add("ProductId", typeof(int));
        itemsDt.Columns.Add("ProductName", typeof(string));
        itemsDt.Columns.Add("Quantity", typeof(int));
        itemsDt.Columns.Add("UnitPrice", typeof(decimal));

        foreach (var item in jArray)
        {
            int productId = item["productId"]?.Value<int>() ?? 0;
            string productName = item["productName"]?.Value<string>()
            int qty = item["quantity"]?.Value<int>() ?? 0;
            decimal unitPrice = item["unitPrice"]?.Value<decimal>() ??
```

```
        if (productId ≤ 0 || qty ≤ 0)
        {
            return Json(new { success = false, message = "Invalid"
        }

        itemsDt.Rows.Add(productId, productName, qty, unitPrice);
    }

    int createdBy = HttpContext.Session.GetInt32("UserId") ?? 1;
    string invoiceNo;
    string errorMessage;

    bool success = _invoiceBAL.CreateInvoice(customerName, custome

    if (success)
    {
        return Json(new { success = true, invoiceNo = invoiceNo, m
    }
    else
    {
        return Json(new { success = false, message = errorMessage
    }
    }
    catch (Exception ex)
    {
        return Json(new { success = false, message = "Error creating i
    }
}

[HttpGet]
public IActionResult History()
{
    if (!IsLoggedIn()) return RedirectToAction("Login", "Account");
    return View();
}

[HttpGet]
public IActionResult GetInvoicesJson()
{
    if (!IsLoggedIn()) return Unauthorized();

    // If Admin, sees all invoices; if Employee, sees all or own
    DataTable dt = _invoiceBAL.GetInvoiceHistory();
    string json = JsonConvert.SerializeObject(dt);
    return Content(json, "application/json");
}
```

```
[HttpGet]
public IActionResult Details(int id)
{
    if (!IsLoggedIn()) return RedirectToAction("Login", "Account");

    DataTable dtHeader = _invoiceBAL.GetInvoiceHeader(id);
    if (dtHeader == null || dtHeader.Rows.Count == 0)
    {
        return NotFound("Invoice not found.");
    }

    DataTable dtDetails = _invoiceBAL.GetInvoiceDetails(id);

    ViewBag.Header = dtHeader.Rows[0];
    return View(dtDetails);
}

[HttpGet]
public IActionResult GetInvoiceDetailsJson(int id)
{
    if (!IsLoggedIn()) return Unauthorized();

    DataTable dtHeader = _invoiceBAL.GetInvoiceHeader(id);
    if (dtHeader == null || dtHeader.Rows.Count == 0)
    {
        string notFoundJson = JsonConvert.SerializeObject(new { succes
        return Content(notFoundJson, "application/json");
    }

    DataTable dtDetails = _invoiceBAL.GetInvoiceDetails(id);

    // Serialize header rows and detail rows manually to avoid DataTab
    var headerRows = new System.Collections.Generic.List<System Collec
    foreach (DataRow r in dtHeader.Rows)
    {
        var dict = new System.Collections.Generic.Dictionary<string, o
        foreach (DataColumn col in dtHeader.Columns)
            dict[col.ColumnName] = r[col] == DBNull.Value ? null : r[c
        headerRows.Add(dict);
    }

    var detailRows = new System.Collections.Generic.List<System Collec
    foreach (DataRow r in dtDetails.Rows)
    {
        var dict = new System.Collections.Generic.Dictionary<string, o
        foreach (DataColumn col in dtDetails.Columns)
```

```

        dict[col.ColumnName] = r[col] == DBNull.Value ? null : r[c
        detailRows.Add(dict);
    }

    string json = JsonConvert.SerializeObject(new { success = true, he
    return Content(json, "application/json");
}
}
}

```

5.2 ProductController.cs

```

using System;
using System.Data;
using System.IO;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Newtonsoft.Json;
using ShopManagement.Business;

namespace ShopManagement.Web.Controllers
{
    public class ProductController : Controller
    {
        private readonly ProductBAL _productBAL = new ProductBAL();
        private readonly ExcelBAL _excelBAL = new ExcelBAL();
        private readonly IWebHostEnvironment _env;

        public ProductController(IWebHostEnvironment env)
        {
            _env = env;
        }

        private bool IsAdmin()
        {
            string? role = HttpContext.Session.GetString("Role");
            return !string.IsNullOrEmpty(role) && role.Equals("Admin", StringC

        [HttpGet]
        public IActionResult Index()
        {
            if (!IsAdmin()) return RedirectToAction("Login", "Account");
        }
    }
}

```

```
        return View();
    }

    [HttpGet]
    public IActionResult GetProductsJson()
    {
        if (string.IsNullOrEmpty(HttpContext.Session.GetString("Role")))
            return Unauthorized();

        DataTable dt = _productBAL.GetAllProducts(true);
        return Content(JsonConvert.SerializeObject(dt), "application/json")
    }

    [HttpGet]
    public IActionResult GetProductByIdJson(int id)
    {
        if (!IsAdmin()) return Unauthorized();
        DataTable dt = _productBAL.GetProductById(id);
        return Content(JsonConvert.SerializeObject(dt), "application/json")
    }

    [HttpGet]
    public IActionResult SearchAvailable(string term)
    {
        DataTable dt = _productBAL.SearchAvailableProducts(term);
        return Content(JsonConvert.SerializeObject(dt), "application/json")
    }

    // -----
    // UPSERT: Add new or increment stock on existing product
    // -----
    [HttpPost]
    public IActionResult Add(string productCode, string productName, int q
    {
        if (!IsAdmin()) return Json(new { success = false, message = "Unau

        if (string.IsNullOrEmpty(productCode) || string.IsNullOrEmpty
            return Json(new { success = false, message = "Product Code and

        string? imagePath = SaveUploadedImage(imageFile);

        string operation;
        bool success = _productBAL.UpsertProduct(productCode, productName,

        return Json(new { success, message = operation });
    }
```

```
// -----  
// EDIT: Update an existing product (exact quantity, price, name, image)  
// -----  
[HttpPost]  
public IActionResult Edit(int productId, string productName, int quantity, decimal price, string image)  
{  
    if (!IsAdmin()) return Json(new { success = false, message = "Unauthorized" });  
  
    if (string.IsNullOrEmpty(productName))  
        return Json(new { success = false, message = "Product Name is required" });  
  
    string? newImagePath = SaveUploadedImage(imageFile);  
  
    string errorMessage;  
    bool success = _productBAL.UpdateProduct(productId, productName, quantity, price, newImagePath, out errorMessage);  
  
    return Json(new { success, message = success ? "Product updated successfully" : errorMessage });  
}  
  
[HttpPost]  
public IActionResult Delete(int productId)  
{  
    if (!IsAdmin()) return Json(new { success = false, message = "Unauthorized" });  
  
    bool success = _productBAL.DeleteProduct(productId);  
    return Json(new { success, message = success ? "Product deleted successfully" : "Error" });  
}  
  
// -----  
// Excel Import  
// -----  
[HttpGet]  
public IActionResult Import()  
{  
    if (!IsAdmin()) return RedirectToAction("Login", "Account");  
    return View();  
}  
  
[HttpPost]  
public IActionResult Import(IFormFile excelFile)  
{  
    if (!IsAdmin()) return RedirectToAction("Login", "Account");  
  
    if (excelFile == null || excelFile.Length == 0)  
    {  
        ViewBag.Error = "Please select a valid Excel file (.xlsx).";  
        return View();  
    }  
}
```

```
}

try
{
    int total, inserted, updated, failed;
    using (var stream = excelFile.OpenReadStream())
    {
        DataTable resultDt = _excelBAL.ProcessExcelProductImport(s

        ViewBag.Total    = total;
        ViewBag.Inserted  = inserted;
        ViewBag.Updated   = updated;
        ViewBag.Failed    = failed;
        ViewBag.Success   = true;

        return View(resultDt);
    }
}
catch (Exception ex)
{
    ViewBag.Error = "Error parsing Excel: " + ex.Message;
    return View();
}
}

[HttpGet]
public IActionResult DownloadTemplate()
{
    byte[] fileBytes = _excelBAL.GenerateSampleExcelTemplate();
    return File(fileBytes, "application/vnd.openxmlformats-officedocum

// -----
// Shared helper: save uploaded image file and return web path
// -----
private string? SaveUploadedImage(IFormFile? imageFile)
{
    if (imageFile == null || imageFile.Length == 0) return null;

    string uploadFolder = Path.Combine(_env.WebRootPath, "uploads", "p
    if (!Directory.Exists(uploadFolder))
        Directory.CreateDirectory(uploadFolder);

    string uniqueFileName = $"{Guid.NewGuid()}_{Path.GetFileName(image
    string filePath = Path.Combine(uploadFolder, uniqueFileName);
    using (var stream = new FileStream(filePath, FileMode.Create))
    {
```



```
        foreach (DataRow r in dt.Rows)
        {
            var d = new System.Collections.Generic.Dictionary<string,
            foreach (DataColumn c in dt.Columns)
            {
                d[c.ColumnName] = r[c] == DBNull.Value ? null : r[c];
            }
            list.Add(d);
        }
        return Content(JsonConvert.SerializeObject(list), "application
    }
    catch (Exception ex)
    {
        return Json(new { error = ex.Message });
    }
}

[HttpPost]
public IActionResult Add(string name, string username, string password
{
    if (!IsAdmin()) return Json(new { success = false, message = "Unau

    if (string.IsNullOrEmpty(name) || string.IsNullOrEmpty(u
    {
        return Json(new { success = false, message = "Please enter Nam

    }

    string errorMessage;
    bool success = _employeeBAL.CreateEmployee(name.Trim(), username.T

    return Json(new { success, message = success ? $"Desk login for '{

}

[HttpPost]
public IActionResult Update(int userId, string name, string? password,
{
    if (!IsAdmin()) return Json(new { success = false, message = "Unau

    string errorMessage;
    bool success = _employeeBAL.UpdateEmployee(userId, name, password,
    return Json(new { success, message = success ? "Desk login updated

}

[HttpPost]
public IActionResult ResetPassword(int userId, string newPassword)
{
    if (!IsAdmin()) return Json(new { success = false, message = "Unau
```

```

        string errorMessage;
        bool success = _employeeBAL.ResetPassword(userId, newPassword, out
        return Json(new { success, message = success ? "Password changed s
    }

    [HttpPost]
    public IActionResult ToggleStatus(int userId)
    {
        if (!IsAdmin()) return Json(new { success = false, message = "Unau

        bool success = _employeeBAL.ToggleStatus(userId);
        return Json(new { success, message = success ? "Desk login status
    }

    [HttpPost]
    public IActionResult Delete(int userId)
    {
        if (!IsAdmin()) return Json(new { success = false, message = "Unau

        bool success = _employeeBAL.DeleteEmployee(userId);
        return Json(new { success, message = success ? "Desk login deactiv
    }
}
}
}

```

5.4 StoreController.cs

```

using System;
using System.Data;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Newtonsoft.Json;
using ShopManagement.Business;

namespace ShopManagement.Web.Controllers
{
    public class StoreController : Controller
    {
        private readonly StoreBAL _storeBAL = new StoreBAL();

        private bool IsAdmin()
        {
            string? role = HttpContext.Session.GetString("Role");

```

```
        return !string.IsNullOrEmpty(role) && role.Equals("Admin", StringComparison.OrdinalIgnoreCase);
    }

    [HttpGet]
    public IActionResult Index()
    {
        if (!IsAdmin()) return RedirectToAction("Login", "Account");
        return View();
    }

    [HttpGet]
    public IActionResult GetStoresJson()
    {
        if (!IsAdmin()) return Unauthorized();
        DataTable dt = _storeBAL.GetAllStores(false);
        return Content(JsonConvert.SerializeObject(dt), "application/json");
    }

    [HttpGet]
    public IActionResult GetActiveStoresJson()
    {
        if (string.IsNullOrEmpty(HttpContext.Session.GetString("Role"))) return Unauthorized();
        DataTable dt = _storeBAL.GetAllStores(activeOnly: true);
        return Content(JsonConvert.SerializeObject(dt), "application/json");
    }

    [HttpGet]
    public IActionResult GetStoreStatsJson()
    {
        if (!IsAdmin()) return Unauthorized();
        DataTable dt = _storeBAL.GetStoreStats();
        return Content(JsonConvert.SerializeObject(dt), "application/json");
    }

    [HttpPost]
    public IActionResult Create(string storeCode, string storeName, string address)
    {
        if (!IsAdmin()) return Json(new { success = false, message = "Unauthorized" });

        string errorMessage;
        bool success = _storeBAL.CreateStore(storeCode, storeName, address);

        return Json(new
        {
            success = success,
            message = success ? $"Store '{storeName}' created successfully" : "Error creating store"
        });
    }
}
```

```

    }

    [HttpPost]
    public IActionResult Update(int storeId, string storeName, string addr
    {
        if (!IsAdmin()) return Json(new { success = false, message = "Unau

        string errorMessage;
        bool success = _storeBAL.UpdateStore(storeId, storeName, address,

        return Json(new { success = success, message = success ? "Store up

    }

    [HttpPost]
    public IActionResult Delete(int storeId)
    {
        if (!IsAdmin()) return Json(new { success = false, message = "Unau

        bool success = _storeBAL.DeleteStore(storeId);
        return Json(new { success = success, message = success ? "Store de

    }
}
}

```

5.5 DashboardController.cs

```

using System;
using System.Data;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using ShopManagement.Business;

namespace ShopManagement.Web.Controllers
{
    public class DashboardController : Controller
    {
        private readonly InvoiceBAL _invoiceBAL = new InvoiceBAL();

        [HttpGet]
        public IActionResult Index()
        {
            string? role = HttpContext.Session.GetString("Role");
            if (string.IsNullOrEmpty(role))
            {

```

```
        return RedirectToAction("Login", "Account");
    }

    if (!role.Equals("Admin", StringComparison.OrdinalIgnoreCase))
    {
        return RedirectToAction("Create", "Invoice");
    }

    DataTable dtMetrics = _invoiceBAL.GetDashboardMetrics();
    return View(dtMetrics);
}
}
```

5.6 AccountController.cs

```
using System;
using System.Data;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using ShopManagement.Business;

namespace ShopManagement.Web.Controllers
{
    public class AccountController : Controller
    {
        private readonly AccountBAL _accountBAL = new AccountBAL();

        [HttpGet]
        public IActionResult Index()
        {
            return RedirectToAction("Login");
        }

        [HttpGet]
        public IActionResult Login()
        {
            string? role = HttpContext.Session.GetString("Role");
            if (!string.IsNullOrEmpty(role))
            {
                return role.Equals("Admin", StringComparison.OrdinalIgnoreCase)
                    ? RedirectToAction("Index", "Product")
                    : RedirectToAction("Create", "Invoice");
            }
            return View();
        }
    }
}
```

```
}

[HttpPost]
public IActionResult Login(string username, string password)
{
    if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))
    {
        ViewBag.Error = "Please enter both username and password.";
        return View();
    }

    DataTable dt = _accountBAL.Login(username, password);

    if (dt != null && dt.Rows.Count > 0)
    {
        DataRow row = dt.Rows[0];
        int userId = Convert.ToInt32(row["UserId"]);
        string fullName = Convert.ToString(row["Name"]) ?? "";
        string user = Convert.ToString(row["Username"]) ?? "";
        string role = Convert.ToString(row["Role"]) ?? "";

        HttpContext.Session.SetInt32("UserId", userId);
        HttpContext.Session.SetString("Username", user);
        HttpContext.Session.SetString("FullName", fullName);
        HttpContext.Session.SetString("Role", role);

        if (role.Equals("Admin", StringComparison.OrdinalIgnoreCase))
        {
            return RedirectToAction("Index", "Product");
        }
        else
        {
            return RedirectToAction("Create", "Invoice");
        }
    }

    ViewBag.Error = "Invalid credentials or account is inactive.";
    return View();
}

[HttpGet]
public IActionResult Logout()
{
    HttpContext.Session.Clear();
    return RedirectToAction("Login");
}
```

```
}  
}
```

JS 6. Standalone Frontend JavaScript (`wwwroot/js/`)

All page-specific JavaScript logic is decoupled into static modular files in `wwwroot/js/` for maximum performance and caching.

6.1 invoice-create.js (POS Billing & Cart Management)

```
// invoice-create.js - New Invoice (POS) page logic  
  
var cart = [];  
var selectedProduct = null;  
  
$(document).ready(function () {  
    initProductDropdown();  
});  
  
function initProductDropdown() {  
    $("#productDropdown").kendoDropDownList({  
        dataTextField: "ProductName",  
        dataValueField: "ProductId",  
        filter: "contains",  
        optionLabel: "-- Choose a product --",  
        template: d => d.ProductId ? `${d.ProductName} [Code: ${d.ProductCode}]` : "",  
        dataSource: {  
            transport: { read: { url: "/Product/GetProductsJson", dataType: "json" } },  
            schema: {  
                parse: function (response) {  
                    return response.filter(x => x.Quantity > 0);  
                }  
            },  
            change: function (e) {  
                var item = this.dataItem();  
                // ...  
            }  
        }  
    });  
}
```

```
        if (item && item.ProductId) {
            selectedProduct = item;
            $("#txtPrice").val(parseFloat(item.Price).toFixed(2));
            $("#numQty").attr("max", item.Quantity).val(1);
        } else {
            selectedProduct = null;
            $("#txtPrice").val("0.00");
        }
    }
});
}

function addItemToCart() {
    if (!selectedProduct) {
        Swal.fire({ icon: 'warning', title: 'Select Product', text: 'Please se
        return;
    }

    var qty = parseInt($("#numQty").val()) || 1;
    if (qty ≤ 0) {
        Swal.fire({ icon: 'warning', title: 'Invalid Qty', text: 'Quantity mus
        return;
    }

    var existing = cart.find(x ⇒ x.productId === selectedProduct.ProductId);
    var maxStock = selectedProduct.Quantity;

    if (existing) {
        if (existing.quantity + qty > maxStock) {
            Swal.fire({ icon: 'warning', title: 'Stock Limit', text: `Cannot a
            return;
        }
        existing.quantity += qty;
    } else {
        if (qty > maxStock) {
            Swal.fire({ icon: 'warning', title: 'Stock Limit', text: `Only ${m
            return;
        }
        cart.push({
            productId: selectedProduct.ProductId,
            productName: selectedProduct.ProductName,
            unitPrice: parseFloat(selectedProduct.Price),
            quantity: qty,
            maxStock: maxStock
        });
    }
}
```

```
renderCart();  
// Reset dropdown  
$("#productDropdown").data("kendoDropDownList").value("");  
selectedProduct = null;  
$("#txtPrice").val("0.00");  
$("#numQty").val(1);  
}  
  
function renderCart() {  
    var $tbody = $("#cartTableBody");  
    $tbody.empty();  
  
    if (cart.length === 0) {  
        $tbody.html('<tr id="emptyCartRow"><td colspan="6" class="text-center  
            $("#lblTotalItems").text("0");  
            $("#lblGrandTotal").text("Rs.0.00");  
            return;  
        }  
  
        var grandTotal = 0;  
        var totalItems = 0;  
  
        cart.forEach((item, index) => {  
            var lineTotal = item.quantity * item.unitPrice;  
            grandTotal += lineTotal;  
            totalItems += item.quantity;  
  
            $tbody.append(`  
                <tr>  
                    <td>${index + 1}</td>  
                    <td>${item.productName}</td>  
                    <td class="text-center">${item.quantity}</td>  
                    <td class="text-end">Rs.${item.unitPrice.toFixed(2)}</td>  
                    <td class="text-end">Rs.${lineTotal.toFixed(2)}</td>  
                    <td class="text-center">  
                        <button type="button" class="btn btn-sm btn-outline-danger  
                    </td>  
                </tr>  
            `);  
        });  
  
        $("#lblTotalItems").text(totalItems);  
        $("#lblGrandTotal").text("Rs." + grandTotal.toFixed(2));  
    }  
  
    function removeItem(index) {  
        cart.splice(index, 1);  
    }  
}
```

```
renderCart();
}

function clearCart() {
  cart = [];
  renderCart();
}

function submitInvoice() {
  var custName = $("#txtCustName").val().trim();
  var custPhone = $("#txtCustPhone").val().trim();

  if (!custName) {
    Swal.fire({ icon: 'warning', title: 'Missing Info', text: 'Please ente
    return;
  }

  if (cart.length === 0) {
    Swal.fire({ icon: 'warning', title: 'Empty Cart', text: 'Please add at
    return;
  }

  $.post("/Invoice/Create", {
    customerName: custName,
    customerPhone: custPhone,
    itemsJson: JSON.stringify(cart)
  }, function (res) {
    if (res.success) {
      Swal.fire({
        icon: 'success',
        title: 'Invoice Created!',
        text: 'Bill No: ' + res.invoiceNo,
        confirmButtonText: 'View History'
      }).then(() => {
        window.location.href = "/Invoice/History";
      });
    } else {
      Swal.fire({ icon: 'error', title: 'Billing Error', text: res.messa
    }
  });
}
```

6.2 employee.js (Staff Grid & Modals)

```
// employee.js - Desk Logins (Staff Accounts) page logic

var addModal, editModal;

$(document).ready(function () {
    addModal = new bootstrap.Modal(document.getElementById('addEmployeeModal')
    editModal = new bootstrap.Modal(document.getElementById('editEmployeeModal')
    loadEmployeesGrid();

    $("#addEmployeeForm").on("submit", function (e) {
        e.preventDefault();
        $.post("/Employee/Add", $(this).serialize(), function (res) {
            if (res.success) {
                Swal.fire({ icon: 'success', title: 'Created', text: res.messa
                addModal.hide();
                $("#addEmployeeForm")[0].reset();
                $("#employeesGrid").data("kendoGrid").dataSource.read();
            } else {
                Swal.fire({ icon: 'error', title: 'Error', text: res.message }
            }
        });
    });

    $("#editEmployeeForm").on("submit", function (e) {
        e.preventDefault();
        $.post("/Employee/Update", $(this).serialize(), function (res) {
            if (res.success) {
                Swal.fire({ icon: 'success', title: 'Updated', text: res.messa
                editModal.hide();
                $("#employeesGrid").data("kendoGrid").dataSource.read();
            } else {
                Swal.fire({ icon: 'error', title: 'Error', text: res.message }
            }
        });
    });
});

function loadEmployeesGrid() {
    $("#employeesGrid").kendoGrid({
        dataSource: {
            transport: { read: { url: "/Employee/GetEmployeesJson", dataType:
            schema: {
                model: {
                    id: "UserId",
                    fields: {
                        UserId: { type: "number" },

```

```

        Name:      { type: "string" },
        Username:  { type: "string" },
        Role:      { type: "string" },
        StoreName: { type: "string" },
        IsActive:  { type: "number" },
        CreatedDate:{ type: "date" }
      }
    },
    pageSize: 10
  },
  pageable: true,
  sortable: true,
  filterable: true,
  columns: [
    { field: "Name", title: "Full Name" },
    { field: "Username", title: "Username", width: 150 },
    { field: "StoreName", title: "Branch / Store", width: 160 },
    {
      field: "IsActive", title: "Status", width: 120,
      template: d => d.IsActive === 1 ? '<span class="badge bg-succe
    },
    {
      title: "Actions", width: 220, filterable: false, sortable: fal
      template: d => `
        <button class="btn btn-sm btn-outline-primary me-1" oncl
        <button class="btn btn-sm btn-outline-secondary me-1" oncl
        <button class="btn btn-sm btn-outline-danger" onclick="del
      `
    }
  ]
});

function openAddModal() {
  $("#addEmployeeForm")[0].reset();
  addModal.show();
}

function openEditModal(userId, name, isActive) {
  $("#edit_userId").val(userId);
  $("#edit_name").val(name);
  $("#edit_isActive").prop("checked", isActive === 1);
  editModal.show();
}

function toggleStatus(userId) {

```

```
$.post("/Employee/ToggleStatus", { userId: userId }, function (res) {
    $("#employeesGrid").data("kendoGrid").dataSource.read();
});
}

function deleteEmployee(userId) {
    Swal.fire({
        title: 'Deactivate Login?',
        icon: 'warning',
        showCancelButton: true,
        confirmButtonText: 'Yes, Deactivate'
    }).then((res) => {
        if (res.isConfirmed) {
            $.post("/Employee/Delete", { userId: userId }, function (r) {
                $("#employeesGrid").data("kendoGrid").dataSource.read();
            });
        }
    });
}
```

6.3 product.js (Product Inventory Grid & Upsert)

```
// product.js - Product Inventory page logic

var upsertModal, editModal;

$(document).ready(function () {
    upsertModal = new bootstrap.Modal(document.getElementById('upsertProductMo
    editModal   = new bootstrap.Modal(document.getElementById('editProductModa
    loadProductsGrid());

    $("#upsertProductForm").on("submit", function (e) {
        e.preventDefault();
        var formData = new FormData(this);
        $.ajax({
            url: "/Product/Add",
            type: "POST",
            data: formData,
            contentType: false,
            processData: false,
            success: function (res) {
                if (res.success) {
                    Swal.fire({ icon: 'success', title: 'Saved', text: res.mes
                    upsertModal.hide();
                }
            }
        });
    });
});
```

```
        $("#upsertProductForm")[0].reset();
        $("#productsGrid").data("kendoGrid").dataSource.read();
    } else {
        Swal.fire({ icon: 'error', title: 'Error', text: res.messa
    }
    }
});
});

$("#editProductForm").on("submit", function (e) {
    e.preventDefault();
    var formData = new FormData(this);
    $.ajax({
        url: "/Product/Edit",
        type: "POST",
        data: formData,
        contentType: false,
        processData: false,
        success: function (res) {
            if (res.success) {
                Swal.fire({ icon: 'success', title: 'Updated', text: res.m
                editModal.hide();
                $("#productsGrid").data("kendoGrid").dataSource.read();
            } else {
                Swal.fire({ icon: 'error', title: 'Error', text: res.messa
            }
        }
    });
});
});

function loadProductsGrid() {
    $("#productsGrid").kendoGrid({
        dataSource: {
            transport: { read: { url: "/Product/GetProductsJson", dataType: "j
            schema: {
                model: {
                    id: "ProductId",
                    fields: {
                        ProductId: { type: "number" },
                        ProductCode: { type: "string" },
                        ProductName: { type: "string" },
                        Quantity: { type: "number" },
                        Price: { type: "number" },
                        ImagePath: { type: "string" }
                    }
                }
            }
        }
    });
}
```

```

        },
        pageSize: 10
    },
    pageable: true,
    sortable: true,
    filterable: true,
    columns: [
        {
            field: "ImagePath", title: "Image", width: 80, filterable: false,
            template: d => ` `<strong>${d.Quantity}</strong>`
        },
        {
            field: "Price", title: "Price (Rs.)", width: 120,
            template: d => `Rs.${parseFloat(d.Price).toFixed(2)}`
        },
        {
            title: "Actions", width: 160, filterable: false, sortable: false,
            template: d => `
                <button class="btn btn-sm btn-outline-primary me-1" onclick="openUpsertModal('${d.ProductId}')">Add</button>
                <button class="btn btn-sm btn-outline-danger" onclick="openEditModal('${d.ProductId}')">Edit</button>
            `
        }
    ]
});
}

function openUpsertModal() {
    $("#upsertProductForm")[0].reset();
    $("#upsert_imgPreview").attr("src", "/images/products/default-product.svg");
    upsertModal.show();
}

function openEditModal(productId) {
    $.getJSON("/Product/GetProductByIdJson?id=" + productId, function (data) {
        if (data && data.length > 0) {
            var item = data[0];
            $("#edit_productId").val(item.ProductId);
            $("#edit_productCode").val(item.ProductCode);
            $("#edit_productName").val(item.ProductName);
            $("#edit_quantity").val(item.Quantity);
            $("#edit_price").val(item.Price);
        }
    });
}

```

```

        $("#edit_imgPreview").attr("src", item.ImagePath || "/images/produ
        editModal.show();
    }
});
}

function deleteProduct(productId, name) {
    Swal.fire({
        title: 'Delete ' + name + '?',
        icon: 'warning',
        showCancelButton: true,
        confirmButtonText: 'Yes, Delete'
    }).then((res) => {
        if (res.isConfirmed) {
            $.post("/Product/Delete", { productId: productId }, function (r) {
                if (r.success) {
                    $("#productsGrid").data("kendoGrid").dataSource.read();
                } else {
                    Swal.fire('Error', r.message, 'error');
                }
            });
        }
    });
}

function previewImage(input, imgId) {
    if (input.files && input.files[0]) {
        var reader = new FileReader();
        reader.onload = e => { document.getElementById(imgId).src = e.target.r
        reader.readAsDataURL(input.files[0]);
    }
}

```

6.4 store.js (Store Branch Management)

```

// store.js - Stores & Branch Management page logic

var createModal, editModal;

$(document).ready(function () {
    createModal = new bootstrap.Modal(document.getElementById('createStoreModa
    editModal   = new bootstrap.Modal(document.getElementById('editStoreModal'
    loadStoresGrid();

```

```
$("#createStoreForm").on("submit", function (e) {
    e.preventDefault();
    $.post("/Store/Create", $(this).serialize(), function (res) {
        if (res.success) {
            Swal.fire({ icon: 'success', title: 'Created', text: res.message });
            createModal.hide();
            $("#createStoreForm")[0].reset();
            $("#storesGrid").data("kendoGrid").dataSource.read();
        } else {
            Swal.fire({ icon: 'error', title: 'Error', text: res.message });
        }
    });
});

$("#editStoreForm").on("submit", function (e) {
    e.preventDefault();
    $.post("/Store/Update", $(this).serialize(), function (res) {
        if (res.success) {
            Swal.fire({ icon: 'success', title: 'Updated', text: res.message });
            editModal.hide();
            $("#storesGrid").data("kendoGrid").dataSource.read();
        } else {
            Swal.fire({ icon: 'error', title: 'Error', text: res.message });
        }
    });
});

function loadStoresGrid() {
    $("#storesGrid").kendoGrid({
        dataSource: {
            transport: { read: { url: "/Store/GetStoresJson", dataType: "json" } },
            schema: {
                model: {
                    id: "StoreId",
                    fields: {
                        StoreId: { type: "number" },
                        StoreCode: { type: "string" },
                        StoreName: { type: "string" },
                        Phone: { type: "string" },
                        Address: { type: "string" },
                        IsActive: { type: "number" }
                    }
                }
            },
            pageSize: 10
        },
    });
}
```

```

        pageable: true,
        sortable: true,
        filterable: true,
        columns: [
            { field: "StoreCode", title: "Code", width: 120 },
            { field: "StoreName", title: "Store Name" },
            { field: "Phone", title: "Phone", width: 140 },
            { field: "Address", title: "Address" },
            {
                field: "IsActive", title: "Status", width: 120,
                template: d => d.IsActive === 1 ? '<span class="badge bg-succe
            },
            {
                title: "Actions", width: 160, filterable: false, sortable: fal
                template: d => `
                    <button class="btn btn-sm btn-outline-primary me-1" oncl
                    <button class="btn btn-sm btn-outline-danger" onclick="del
                `
            }
        ]
    });
}

function openCreateModal() {
    $("#createStoreForm")[0].reset();
    createModal.show();
}

function openEditModal(id, name, phone, address, isActive) {
    $("#edit_storeId").val(id);
    $("#edit_storeName").val(name);
    $("#edit_phone").val(phone);
    $("#edit_address").val(address);
    $("#edit_isActive").prop("checked", isActive === 1);
    editModal.show();
}

function deleteStore(storeId) {
    Swal.fire({
        title: 'Deactivate Store?',
        icon: 'warning',
        showCancelButton: true,
        confirmButtonText: 'Yes, Deactivate'
    }).then((res) => {
        if (res.isConfirmed) {
            $.post("/Store/Delete", { storeId: storeId }, function (r) {
                $("#storesGrid").data("kendoGrid").dataSource.read();
            });
        }
    });
}

```

```
});  
}  
});  
}
```

6.5 invoice-history.js (Invoice History & Printing)

```
// invoice-history.js - Invoice History page logic  
  
var detailsModal;  
  
$(document).ready(function () {  
    detailsModal = new bootstrap.Modal(document.getElementById('invoiceDetails  
    loadInvoicesGrid();  
});  
  
function loadInvoicesGrid() {  
    $("#invoicesGrid").kendoGrid({  
        dataSource: {  
            transport: { read: { url: "/Invoice/GetInvoicesJson", dataType: "j  
            schema: {  
                model: {  
                    id: "InvoiceId",  
                    fields: {  
                        InvoiceId: { type: "number" },  
                        InvoiceNo: { type: "string" },  
                        CustomerName: { type: "string" },  
                        CustomerPhone: { type: "string" },  
                        TotalAmount: { type: "number" },  
                        InvoiceDate: { type: "date" },  
                        CreatedByName: { type: "string" }  
                    }  
                }  
            },  
            pageSize: 10  
        },  
        pageable: true,  
        sortable: true,  
        filterable: true,  
        columns: [  
            { field: "InvoiceNo", title: "Invoice No", width: 160 },  
            { field: "CustomerName", title: "Customer Name" },  
            { field: "CustomerPhone", title: "Phone", width: 140 },  
            { field: "CreatedByName", title: "Cashier", width: 140 },
```

```

        {
            field: "InvoiceDate", title: "Date", width: 170,
            template: d => kendo.toString(kendo.parseDate(d.InvoiceDate),
        },
        {
            field: "TotalAmount", title: "Total (Rs.)", width: 130,
            template: d => `Rs.${parseFloat(d.TotalAmount).toFixed(2)}`
        },
        {
            title: "Actions", width: 110, filterable: false, sortable: false,
            template: d => `

```

```
function printInvoiceModal() {
    var printContents = document.getElementById("printableInvoiceArea").innerHTML;
    var win = window.open('', '', 'height=650,width=800');
    win.document.write('<html><head><title>Print Bill</title>');
    win.document.write('<link rel="stylesheet" href="https://cdn.jsdelivr.net/'
    win.document.write('</head><body class="p-4">');
    win.document.write(printContents);
    win.document.write('</body></html>');
    win.document.close();
    win.focus();
    setTimeout(() => { win.print(); win.close(); }, 500);
}
```

7. Razor Views & Script Integration

7.1 Views/Invoice/Create.cshtml

```
@{
    ViewData["Title"] = "New Customer Invoice (POS)";
    Layout = "_Layout";
}

<div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="mb-0">New Invoice (Billing POS)</h4>
    <a href="/Invoice/History" class="btn btn-outline-secondary">Invoice Histo
</div>

<div class="row g-3">
    <!-- Left Column: Customer details & product selector -->
    <div class="col-md-5">
        <div class="border p-3 rounded mb-3 bg-white">
            <h6 class="fw-bold mb-3">1. Customer Information</h6>
            <div class="mb-3">
                <label class="form-label fw-semibold">Customer Name *</label>
                <input type="text" class="form-control" id="txtCustName" place
            </div>
            <div class="mb-2">
                <label class="form-label fw-semibold">Phone Number</label>
```

```

        <input type="text" class="form-control" id="txtCustPhone" plac
    </div>
</div>

<div class="border p-3 rounded bg-white">
    <h6 class="fw-bold mb-3">2. Add Product to Bill</h6>
    <div class="mb-3">
        <label class="form-label fw-semibold">Search / Select Product
        <input id="productDropdown" style="width: 100%;" />
    </div>
    <div class="row g-2 mb-3">
        <div class="col-6">
            <label class="form-label fw-semibold">Quantity *</label>
            <input type="number" class="form-control" id="numQty" valu
        </div>
        <div class="col-6">
            <label class="form-label fw-semibold">Unit Price (₹)</labe
            <input type="text" class="form-control bg-light" id="txtPr
        </div>
    </div>
    <button type="button" class="btn btn-primary w-100" onclick="addIt
</div>
</div>

<!-- Right Column: Cart items table & Billing summary -->
<div class="col-md-7">
    <div class="border p-3 rounded bg-white">
        <div class="d-flex justify-content-between align-items-center mb-3"
            <h6 class="fw-bold mb-0">3. Invoice Items Cart</h6>
            <button type="button" class="btn btn-sm btn-outline-secondary"
        </div>

        <table class="table table-bordered align-middle">
            <thead class="table-light">
                <tr>
                    <th>#</th>
                    <th>Product</th>
                    <th class="text-center" style="width:90px;">Qty</th>
                    <th class="text-end" style="width:110px;">Price</th>
                    <th class="text-end" style="width:110px;">Total</th>
                    <th style="width:50px;"></th>
                </tr>
            </thead>
            <tbody id="cartTableBody">
                <tr id="emptyCartRow">
                    <td colspan="6" class="text-center text-muted py-4">No
                </td>
            </tr>
        </tbody>
    </table>
    </div>

```

```

        </tbody>
      </table>

      <div class="d-flex justify-content-between align-items-center pt-2">
        <div>
          Total Items: <strong id="lblTotalItems">0</strong>
        </div>
        <div class="text-end">
          <span class="fs-5 me-2">Grand Total:</span>
          <strong class="fs-4 text-primary" id="lblGrandTotal">₹0.00
        </div>
      </div>

      <button type="button" class="btn btn-success w-100 mt-3 py-2 fs-5">
        Confirm & Generate Invoice
      </button>
    </div>
  </div>
</div>

@section Scripts {
  <script src="~/js/invoice-create.js"></script>
}

```

7.2 Views/Employee/Index.cshtml

```

@{
    ViewData["Title"] = "Desk Logins";
    Layout = "_Layout";
}

<div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="mb-0">Desk Logins (Staff Accounts)</h4>
    <button type="button" class="btn btn-primary" onclick="openAddModal()">+ C
</div>

<div id="employeesGrid"></div>

<!-- Add Modal -->
<div class="modal fade" id="addEmployeeModal" tabindex="-1">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">

```

```

        <h5 class="modal-title">New Desk Login</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"
    </div>
    <form id="addEmployeeForm">
        <div class="modal-body">
            <div class="mb-3">
                <label class="form-label fw-semibold">Staff Full Name
                <input type="text" name="name" class="form-control" re
            </div>
            <div class="mb-3">
                <label class="form-label fw-semibold">Username *</labe
                <input type="text" name="username" class="form-control
            </div>
            <div class="mb-3">
                <label class="form-label fw-semibold">Password *</labe
                <input type="password" name="password" class="form-con
            </div>
        </div>
        <div class="modal-footer">
            <button type="button" class="btn btn-secondary" data-bs-di
            <button type="submit" class="btn btn-primary">Create Login
        </div>
    </form>
</div>
</div>
</div>

<!-- Edit Modal -->
<div class="modal fade" id="editEmployeeModal" tabindex="-1">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Edit Desk Login</h5>
                <button type="button" class="btn-close" data-bs-dismiss="modal"
            </div>
            <form id="editEmployeeForm">
                <input type="hidden" name="userId" id="edit_userId" />
                <div class="modal-body">
                    <div class="mb-3">
                        <label class="form-label fw-semibold">Staff Full Name
                        <input type="text" name="name" id="edit_name" class="f
                    </div>
                    <div class="mb-3">
                        <label class="form-label fw-semibold">New Password (le
                        <input type="password" name="password" class="form-con
                    </div>
                    <div class="form-check mb-3">

```

```

        <input type="checkbox" name="isActive" id="edit_isActi
        <label class="form-check-label" for="edit_isActive">Ac
    </div>
</div>
<div class="modal-footer">
    <button type="button" class="btn btn-secondary" data-bs-di
    <button type="submit" class="btn btn-primary">Update Login
</div>
</form>
</div>
</div>
</div>

@section Scripts {
    <script src="~/js/employee.js"></script>
}

```

7.3 Views/Product/Index.cshtml

```

@{
    ViewData["Title"] = "Product Inventory";
    Layout = "_Layout";
}

<div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="mb-0">Product Inventory</h4>
    <div>
        <a href="/Product/Import" class="btn btn-outline-secondary me-2">Excel
        <button type="button" class="btn btn-primary" onclick="openUpsertModal
    </div>
</div>

<div id="productsGrid"></div>

<!-- Add / Upsert Modal -->
<div class="modal fade" id="upsertProductModal" tabindex="-1">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Add / Upsert Product</h5>
                <button type="button" class="btn-close" data-bs-dismiss="modal
            </div>
            <form id="upsertProductForm" enctype="multipart/form-data">

```

```

<div class="modal-body">
  <div class="mb-3">
    <label class="form-label fw-semibold">Product Code / S
    <input type="text" class="form-control" name="productC
    <div class="form-text">Agar code pehle se hai toh stoc
  </div>
  <div class="mb-3">
    <label class="form-label fw-semibold">Product Name *</
    <input type="text" class="form-control" name="productN
  </div>
  <div class="row g-2 mb-3">
    <div class="col-6">
      <label class="form-label fw-semibold">Quantity *</
      <input type="number" class="form-control" name="qu
    </div>
    <div class="col-6">
      <label class="form-label fw-semibold">Price (₹) *<
      <input type="number" step="0.01" class="form-contr
    </div>
  </div>
  <div class="mb-3">
    <label class="form-label fw-semibold">Image (Optional)
    <input type="file" class="form-control" name="imageFil
    <div class="mt-2 text-center">
      
  <button type="button" class="btn btn-secondary" data-bs-di
  <button type="submit" class="btn btn-primary">Save Product
</div>
</form>
</div>
</div>
</div>

<!-- Edit Modal -->
<div class="modal fade" id="editProductModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Edit Product Details</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal
      </div>
      <form id="editProductForm" enctype="multipart/form-data">
        <input type="hidden" name="productId" id="edit_productId" />

```

```

<div class="modal-body">
  <div class="mb-3">
    <label class="form-label fw-semibold">Product Code (Re
    <input type="text" class="form-control bg-light" id="e
  </div>
  <div class="mb-3">
    <label class="form-label fw-semibold">Product Name *</
    <input type="text" class="form-control" name="productN
  </div>
  <div class="row g-2 mb-3">
    <div class="col-6">
      <label class="form-label fw-semibold">Exact Quanti
      <input type="number" class="form-control" name="qu
    </div>
    <div class="col-6">
      <label class="form-label fw-semibold">Price (₹) *<
      <input type="number" step="0.01" class="form-contr
    </div>
  </div>
  <div class="mb-3">
    <label class="form-label fw-semibold">Change Image (Op
    <input type="file" class="form-control" name="imageFil
    <div class="mt-2 text-center">
      
  <button type="button" class="btn btn-secondary" data-bs-di
  <button type="submit" class="btn btn-primary">Update Produ
</div>
</form>
</div>
</div>
</div>

@section Scripts {
  <script src="~/js/product.js"></script>
}

```

7.4 Views/Store/Index.cshtml

```

@{
    ViewData["Title"] = "Stores / Branches";
}

```

```

    Layout = "_Layout";
}

<div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="mb-0">Stores & Branch Management</h4>
    <button type="button" class="btn btn-primary" onclick="openCreateModal()">
</div>

<div id="storesGrid"></div>

<!-- Create Modal -->
<div class="modal fade" id="createStoreModal" tabindex="-1">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Create Store Branch</h5>
                <button type="button" class="btn-close" data-bs-dismiss="modal"
            </div>
            <form id="createStoreForm">
                <div class="modal-body">
                    <div class="mb-3">
                        <label class="form-label fw-semibold">Store Code *</label>
                        <input type="text" class="form-control" name="storeCod
                    </div>
                    <div class="mb-3">
                        <label class="form-label fw-semibold">Store Name *</label>
                        <input type="text" class="form-control" name="storeNam
                    </div>
                    <div class="mb-3">
                        <label class="form-label fw-semibold">Phone</label>
                        <input type="text" class="form-control" name="phone" p
                    </div>
                    <div class="mb-3">
                        <label class="form-label fw-semibold">Address</label>
                        <input type="text" class="form-control" name="address"
                    </div>
                </div>
                <div class="modal-footer">
                    <button type="button" class="btn btn-secondary" data-bs-di
                    <button type="submit" class="btn btn-primary">Save Store</
                </div>
            </form>
        </div>
    </div>
</div>

<!-- Edit Modal -->

```

```

<div class="modal fade" id="editStoreModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Edit Store Branch</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" />
      </div>
      <form id="editStoreForm">
        <input type="hidden" name="storeId" id="edit_storeId" />
        <div class="modal-body">
          <div class="mb-3">
            <label class="form-label fw-semibold">Store Name *</label>
            <input type="text" class="form-control" name="storeName" />
          </div>
          <div class="mb-3">
            <label class="form-label fw-semibold">Phone</label>
            <input type="text" class="form-control" name="phone" />
          </div>
          <div class="mb-3">
            <label class="form-label fw-semibold">Address</label>
            <input type="text" class="form-control" name="address" />
          </div>
          <div class="form-check mb-3">
            <input type="checkbox" class="form-check-input" name="isActive" />
            <label class="form-check-label" for="edit_isActive">Active</label>
          </div>
        </div>
        <div class="modal-footer">
          <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Cancel</button>
          <button type="submit" class="btn btn-primary">Update Store</button>
        </div>
      </form>
    </div>
  </div>
</div>

@section Scripts {
  <script src="~/js/store.js"></script>
}

```

7.5 Views/Dashboard/Index.cshtml

```

@using System.Data
@model DataTable
@{
    ViewData["Title"] = "Dashboard Overview";
    Layout = "_Layout";

    long totalProducts = 0;
    long lowStock = 0;
    long totalInvoices = 0;
    decimal totalRevenue = 0;
    long totalEmployees = 0;

    if (Model != null && Model.Rows.Count > 0)
    {
        var row = Model.Rows[0];
        totalProducts = Convert.ToInt64(row["TotalProducts"]);
        lowStock = Convert.ToInt64(row["LowStockProducts"]);
        totalInvoices = Convert.ToInt64(row["TotalInvoices"]);
        totalRevenue = Convert.ToDecimal(row["TotalRevenue"]);
        totalEmployees = Convert.ToInt64(row["TotalEmployees"]);
    }
}

<div class="row g-3 mb-4">
    <!-- Total Products -->
    <div class="col-12 col-sm-6 col-xl-3">
        <div class="card card-custom p-3 h-100">
            <div class="d-flex align-items-center justify-content-between">
                <div>
                    <span class="text-muted small fw-medium text-uppercase">To
                    <h3 class="fw-bold mt-1 mb-0 text-dark">@totalProducts</h3>
                </div>
                <div class="p-3 rounded-3" style="background-color: #e0e7ff; color: #000000;">
                    <i class="fa-solid fa-boxes-stacked fa-xl"></i>
                </div>
            </div>
            <div class="mt-3">
                <a href="/Product" class="small text-primary text-decoration-none">
                    View Products <i class="fa-solid fa-arrow-right ms-1"></i>
                </a>
            </div>
        </div>
    </div>

    <!-- Low Stock Alert -->
    <div class="col-12 col-sm-6 col-xl-3">

```

```

<div class="card card-custom p-3 h-100">
  <div class="d-flex align-items-center justify-content-between">
    <div>
      <span class="text-muted small fw-medium text-uppercase">Lo
      <h3 class="fw-bold mt-1 mb-0 text-warning">@LowStock</h3>
    </div>
    <div class="p-3 rounded-3" style="background-color: #fef3c7; c
      <i class="fa-solid fa-triangle-exclamation fa-xl"></i>
    </div>
  </div>
  <div class="mt-3">
    <span class="text-muted small">&le; 10 units remaining</span>
  </div>
</div>

<!-- Total Invoices -->
<div class="col-12 col-sm-6 col-xl-3">
  <div class="card card-custom p-3 h-100">
    <div class="d-flex align-items-center justify-content-between">
      <div>
        <span class="text-muted small fw-medium text-uppercase">In
        <h3 class="fw-bold mt-1 mb-0 text-dark">@totalInvoices</h3>
      </div>
      <div class="p-3 rounded-3" style="background-color: #dcfce7; c
        <i class="fa-solid fa-file-invoice-dollar fa-xl"></i>
      </div>
    </div>
    <div class="mt-3">
      <a href="/Invoice/History" class="small text-success text-deco
        View History <i class="fa-solid fa-arrow-right ms-1"></i>
      </a>
    </div>
  </div>
</div>

<!-- Total Revenue -->
<div class="col-12 col-sm-6 col-xl-3">
  <div class="card card-custom p-3 h-100">
    <div class="d-flex align-items-center justify-content-between">
      <div>
        <span class="text-muted small fw-medium text-uppercase">To
        <h3 class="fw-bold mt-1 mb-0 text-dark">₹@totalRevenue.ToS
      </div>
      <div class="p-3 rounded-3" style="background-color: #f3e8ff; c
        <i class="fa-solid fa-indian-rupee-sign fa-xl"></i>
      </div>
    </div>
  </div>
</div>

```

```

        </div>
        <div class="mt-3">
            <span class="text-muted small">Active Employees: <strong>@tota
        </div>
    </div>
</div>

<!-- Quick Action Shortcuts -->
<div class="card card-custom p-4 mb-4">
    <h5 class="fw-bold mb-3"><i class="fa-solid fa-bolt me-2 text-primary"></i>
    <div class="row g-3">
        <div class="col-md-3">
            <a href="/Invoice/Create" class="btn btn-primary w-100 py-3 d-flex"
                <i class="fa-solid fa-receipt fa-2x"></i>
                <span class="fw-semibold">Generate New Invoice</span>
            </a>
        </div>
        <div class="col-md-3">
            <a href="/Product" class="btn btn-outline-primary w-100 py-3 d-fle"
                <i class="fa-solid fa-box-open fa-2x"></i>
                <span class="fw-semibold">Add / Update Product</span>
            </a>
        </div>
        <div class="col-md-3">
            <a href="/Product/Import" class="btn btn-outline-success w-100 py-"
                <i class="fa-solid fa-file-excel fa-2x"></i>
                <span class="fw-semibold">Excel Bulk Import</span>
            </a>
        </div>
        <div class="col-md-3">
            <a href="/Employee" class="btn btn-outline-dark w-100 py-3 d-flex"
                <i class="fa-solid fa-user-plus fa-2x"></i>
                <span class="fw-semibold">Create Desk Login</span>
            </a>
        </div>
    </div>
</div>

```

8. Architectural Decisions & Concurrency Design

1. Why Stored Procedures Exclusively?

Centralized schema governance, pre-compiled execution plans in MySQL memory, and zero exposure to SQL injection vulnerabilities.

2. Why Pessimistic Locking (FOR UPDATE) in Invoicing?

Prevents negative inventory in busy retail environments when multiple cashiers sell the last units of an item simultaneously.

3. Why Modular JavaScript in wwwroot/js/?

Allows browser caching, complies with Content Security Policies (CSP), and keeps Razor views clean and focused on markup.

▶ 9. Step-by-Step Setup, Build & Run Guide

1. **Deploy Database:** Open MySQL CLI or Workbench and run

```
ShopManagement_Full_Setup.sql .
```

2. **Verify Connection:** Ensure `ShopManagement.Web/appsettings.json` has correct credentials.

3. **Build Solution:** Run `dotnet build ShopManagement.slnx -c Release`.

4. **Start Web Application:** Run `dotnet run --project ShopManagement.Web`.

5. **Open Browser:** Access `http://localhost:5200` .